

Exercícios — Aula 15 — BFS e uso de filas

Técnicas de Programação - 2026/02

Última atualização: July 26, 2026

Instruções

- BFS usa fila: quem é descoberto primeiro deve ser processado primeiro.
- A matriz ou array `dist` também serve como marca de visitado quando `-1` significa “ainda não visitado”.
- Em toda simulação, acompanhe fila, distâncias e ordem de descoberta.

Exercício 1 — Simulação da fila na BFS

Considere o grafo não direcionado:

```
adj[0] = [1, 2]
adj[1] = [0, 3, 4]
adj[2] = [0, 4]
adj[3] = [1, 5]
adj[4] = [1, 2, 5]
adj[5] = [3, 4]
```

Simule `BFS(adj, 0)`.

Algorithm 1: BfsDistancias

Result: Executa BFS-DISTANCIAS.

Input : list de adjacência `adj`, int origem

Output: array `dist`

```
dist ← ARRAY-PREENCHIDO(TAMANHO(adj), -1)
fila ← FILA-VAZIA()
dist[origem] ← 0
ENFILEIRAR(fila, origem)
while VAZIA(fila) = false do
    atual ← DESENFILEIRAR(fila)
    foreach vizinho ∈ adj[atual] do
        if dist[vizinho] = -1 then
            dist[vizinho] ← dist[atual] + 1
            ENFILEIRAR(fila, vizinho)
        end
    end
end
return dist
```

Preencha uma linha a cada remoção da fila.

passo	removido da fila	vizinhos descobertos	fila ao final do passo	dist ao final do passo
1				
2				
3				
4				
5				
6				

Qual é a distância final de 0 até 5?

Exercício 2 — Matriz de distâncias

No labirinto abaixo, S é origem, D é destino, . é livre e # é parede.

```
#####
#S...#
###.#D#
#.....#
#####
```

Use BFS com vizinhança em 4 direções e preencha a matriz de distâncias. Use # para paredes e -1 para células livres inalcançáveis.

```
#####
#0????#
###?#?#
#?????#
#####
```

Depois responda:

1. Qual é a menor distância de S até D?
2. Quais células entram na fila com distância 1?
3. Quais células entram na fila com distância 2?

Exercício 3 — Reconstrução de caminho

Durante a BFS, além de `dist`, guardamos `pred[vizinho]` \gets `atual` quando descobrimos um vizinho.

Considere:

```
pred[0] = -1
pred[1] = 0
pred[2] = 0
pred[3] = 1
pred[4] = 2
pred[5] = 4
```

Reconstrua o caminho da origem 0 até o destino 5.

1. Comece em 5 e siga predecessores até -1.
2. Escreva o caminho encontrado de trás para frente.
3. Inverta a ordem para obter o caminho da origem até o destino.
4. Qual distância esse caminho sugere?

Agora diga o que deve acontecer se `pred[destino] = -1` e `destino` não é a origem.

Exercício 4 — Menor distância em matriz

Escreva pseudocódigo para MENOR-DISTANCIA-MATRIZ.

Contrato:

- entrada: matriz `lab`, origem `(li, ci)`, destino `(lf, cf)`;
- `#` representa parede e `.` representa livre;
- saída: menor número de passos ou `-1` se não houver caminho;
- use BFS com TAD Fila;
- use uma matriz `dist` inicializada com `-1`.

Inclua no pseudocódigo:

- validação de coordenadas;
- teste de parede;
- geração de vizinhos em quatro direções;
- retorno quando o destino for removido da fila ou ao final da busca.

Exercício 5 — Por que BFS encontra menor caminho?

Explique com suas palavras por que BFS encontra menor caminho em grafos sem peso.

Use obrigatoriamente as ideias:

- fila;
- camadas;
- distância 0, 1, 2, ...;
- primeira vez que um vértice é descoberto.

Depois compare com DFS:

1. DFS pode encontrar um caminho?
2. DFS garante o menor caminho?
3. O que muda se trocarmos a fila por pilha?

Exercício 6 — Quando BFS simples não basta?

Para cada problema, diga se BFS simples resolve ou não.

Problema	BFS simples resolve?	Justificativa
Menor número de portas entre salas, cada porta custa 1		
Menor caminho em labirinto sem pesos		
Caminho de menor custo quando cada estrada tem pedágio diferente		
Quantidade mínima de movimentos em tabuleiro, todos com mesmo custo		
Caminho que maximiza pontuação coletada		
Verificar se existe algum caminho		

Quando BFS simples não resolve, que informação do problema quebra a hipótese de “cada passo custa igual”?

Créditos e reaproveitamento

Exercícios adaptados de labirintos, contraste entre DFS e BFS e simulação de vizinhos dos handouts antigos devem indicar:

Adaptado de material de Igor Montagner para a disciplina Técnicas de Programação.

Fonte externa opcional para variações conceituais: Princeton Undirected Graphs, <https://algs4.cs.princeton.edu/41graph/>.